PROJECT PROPOSAL

CS3203: Software Engineering Project

FreshLens

AI-Powered Automated Inventory and Freshness Monitoring System for Small-Scale Retailers

Group Number : 21
Project ID (PID) : 5
Project Mentor : Mr. Kavindu Rajapaksha

No.	Name	Index Number
1	SENPURA H. H. Y. B.	230602E
2	SENEVIRATHNE U. S. M.	230600V
3	SATHURSHNA U.	230592U

Department of Computer Science and Engineering

July 2026

Contribution of Group Members

Each member of the group contributed to the research, design, and writing of this proposal as summarised below.

Member	Contribution
SENPURA H. H. Y. B. (230602E)	Led problem definition and needs analysis (Sections 1, 3, 4); researched existing inventory-management systems and freshness-detection literature (Section 7); defined risks and limitations; compiled IEEE references.
SENEVIRATHNE U. S. M. (230600V)	Defined system scope, user roles, and the Version 1 / Version 2 split, including the multi-tenant vendor/admin separation (Section 5); drafted deliverables tied to course milestones (Section 6); evaluated CNN architectures for the Two-Tier Classifier.
SATHURSHNA U. (230592U)	Authored the project overview, scan flow, and tenant-isolation summary (Section 2); led the technology feasibility analysis and full stack justification (Section 7); coordinated formatting and final review.

All three members participated jointly in group discussions, proposal review meetings, and the final proofreading of this document.

1. Title of the Project

FreshLens: AI-Powered Automated Inventory and Freshness Monitoring System for Small-Scale Retailers.

2. Overview of the Project

Small-scale grocery retailers and "mom-and-pop" shops typically rely on manual stock counts and visual inspection to manage perishable produce. Without the barcode scanners, smart scales, or IoT sensors available to large supermarket chains, these vendors have no reliable, low-cost way to track how much stock they have or how quickly it is spoiling. FreshLens turns a standard smartphone camera into the primary sensor for both inventory tracking and quality inspection.

The system is a full-stack, multi-tenant SaaS platform supporting two user roles: the Vendor, who uses a mobile application to photograph produce and view their own dashboard, and the Platform Administrator, who uses a web application to manage product catalogues, vendor accounts, and platform-wide analytics. Every table in the database carries a tenant identifier, enforced by PostgreSQL Row-Level Security so that one vendor's data can never be read by another vendor's session: this isolation is enforced at the database layer, not only in application code.

A scan follows a fixed, asynchronous pipeline: the vendor photographs one product type, the image is stored in object storage and a job is queued; the API returns immediately rather than waiting for the model; a background worker runs the image through FreshLens's Two-Tier Classifier (Section 7) and writes the result back to the database; the dashboard updates once the result is ready. The CNN never runs inline inside a web request: this keeps the platform responsive regardless of how long inference takes.

Input to the system is a single-product shelf image plus the vendor's tenant context and a vendor-confirmed quantity. Output is a structured inventory record: item identity, freshness classification, quantity, and a confidence score, surfaced on both the web dashboard and mobile app, with automated alerts for low stock and produce that has been unsold for an unusually long time (Section 5).

3. Objectives of the Project

The objectives of this project are to:

- Design a centralised web and mobile architecture for real-time inventory management.
- Implement a full-stack system capable of securely handling multi-tenant business data, so that each vendor's inventory remains isolated from every other vendor on the platform.
- Develop and integrate a two-tier Deep Learning pipeline that identifies produce type and classifies its freshness, trained using public datasets.
- Evaluate the accuracy of mobile-based visual inspection against manual, human-performed stock logging.
- Establish a clearly scoped Version 1 / Version 2 roadmap so the graded prototype stays achievable while more ambitious inventory-intelligence features remain a documented, feasible future direction.

4. The Need for the Project

Managing perishable stock is a high-stakes, recurring task for small vendors, and the shortcomings of the current manual approach create four compounding problems.

- **Manual tracking errors:** hand-counted stock records are frequently inaccurate, leading to both stockouts and excess ordering.
- **Quality degradation:** without a systematic freshness check, spoiled items remain on display, directly damaging customer trust and repeat business.
- **Technical barriers:** existing Enterprise Resource Planning (ERP) inventory systems are built for large retailers and are too complex and expensive for a micro-business to adopt.
- **Information asymmetry:** vendors have no structured, historical data on which products spoil fastest under their specific shop conditions, so purchasing decisions remain guesswork.

A smartphone-camera-based system removes the hardware cost barrier entirely: the vendor uses a device they already own: while giving them the same class of data-driven insight that larger competitors already have through dedicated IoT and ERP infrastructure.

5. Scope of the Project

The project delivers a complete software ecosystem with no dependency on external hardware sensors. To keep the graded, single-semester deliverable achievable while still documenting a credible growth path, scope is explicitly split into a **Version 1** (this semester's implementation) and a **Version 2** (future work, not part of this semester's graded deliverable).

5.1 User Roles

- **Vendor / Shop Owner**: uses the mobile application to capture shelf images, confirm quantities, view their own inventory and freshness dashboard, and receive alerts.
- **Platform Administrator**: uses the web application to manage product catalogues and vendor profiles, and to view aggregated analytics across the platform.

5.2 Version 1 Scope (This Semester)

- **Mobile application:** camera interface capturing one product type per photo; vendor confirms/enters quantity at scan time; real-time dashboard showing inventory levels and freshness scores.
- **FreshLens Two-Tier Classifier** (Section 7): Tier 1 identifies the produce type; Tier 2 classifies its condition as Fresh, Medium, or Spoiled.
- **Web application:** administrative panel for product catalogues and vendor profiles; analytics engine reporting on waste/spoilage patterns.
- **Backend services:** RESTful API (FastAPI) connecting mobile/web clients to the database; Supabase Auth for tenant-aware authentication; Cloudflare R2 for image storage.
- **Asynchronous inference pipeline:** Celery + Redis queue decoupling CNN inference from the API request cycle, as described in Section 2.
- **Multi-tenant data isolation** via PostgreSQL Row-Level Security on every tenant-scoped table.
- **Low-stock alerts:** triggered when a product's recorded quantity falls below a vendor-configurable threshold.
- **Static "aging" alerts:** each product category has an administrator-configurable typical shelf-life (in days); an alert is raised when a batch's time-since-intake exceeds that figure while a large proportion of the received quantity remains unsold. This is a lookup-table approximation, not a learned model: the learned version is Version 2 (below).
- **Data model** includes a `batches` table (intake date, quantity received, quantity

remaining, category shelf-life) so that Version 2's learned features can be added later without a schema redesign.

5.3 Version 2 Scope (Future Work; Not Graded This Semester)

These features were considered during planning and are technically credible, but each depends on data or models that do not yet exist and would put the graded Version 1 deliverable at risk if attempted this semester. They are documented here to show the platform's intended growth path.

- **Image-based age estimation:** predicting a produce item's age (days since harvest/intake) directly from a photograph. No public dataset labels produce by exact age; this requires a custom-collected, time-labelled dataset before any model can be trained.
- **Per-batch predicted rot-date with uncertainty:** estimating not just current freshness but the date a batch is likely to spoil, accounting for variation in individual items caused by store conditions (temperature, humidity, handling). This is a time-to-event prediction problem layered on top of age estimation, and is not solvable until Version 2's age-estimation model exists.
- **Periodic batch re-evaluation:** re-photographing a bunch of items already in stock to refine its predicted rot-date as more information becomes available. Depends entirely on the two features above.
- **ML-based "nearing spoilage" alerts:** replacing Version 1's static shelf-life lookup table with a learned, per-batch prediction once the age-estimation model exists.
- **Voice-based sale deduction:** a vendor states an item and quantity sold; speech recognition and simple intent parsing deduct it from recorded stock automatically. This is comparatively low-risk compared to the other Version 2 items: it uses existing speech-to-text APIs rather than novel model training: and may be pulled forward into Version 1 if development time allows, but is not committed to the graded deliverable.
- **Automatic multi-item shelf detection:** identifying and counting several different produce types within a single photograph (true object detection, e.g. a YOLO-based pipeline as in Mukhiddinov et al. [1]), replacing Version 1's one-product-per-photo constraint.

Production Scaling Roadmap. Beyond new features, a companion Enterprise Architecture Plan documents how this platform would scale from the Version 1 prototype to a production system serving over one million vendors, under the guiding principle "architect for a million, build for the demo": every Version 1 decision is chosen so that scaling later means adding workers and replicas, not rewriting. Key elements of that roadmap include:

- GPU-autoscaled ML serving with request batching (Triton/TorchServe), replacing Version 1's single inference worker and scaling automatically with queue depth.
- Kubernetes with GitOps deployment (ArgoCD), replacing Version 1's Docker Compose setup, with automatic rollback and self-healing.
- A full observability stack (OpenTelemetry, Prometheus, Loki, Tempo, Grafana) providing per-tenant metrics, structured logs, and distributed tracing beyond Version 1's basic logging.
- Database scaling via read replicas and, eventually, tenant-hash sharding once a single Postgres primary becomes a write bottleneck.
- A tenant-isolation graduation path from Version 1's shared-database Pool model to schema-per-tenant (Bridge) or database-per-tenant (Silo) isolation for larger or compliance-sensitive vendors.
- A staged rollout plan: prototype, launch, growth, scale: tied to concrete vendor-count thresholds rather than an open-ended aspiration.

Full architecture diagrams and a phased implementation checklist are provided in the accompanying Enterprise Architecture Plan.

5.4 Out of Scope (All Versions)

- Integration with physical IoT sensors or smart scales.
- Automated procurement or integration with external supply-chain/logistics systems.
- Financial accounting and tax-filing modules.

6. Deliverables

Deliverables are tied directly to the official CS3203 batch-23 submission milestones:

- Project Proposal (this document).
- Feasibility Study Document and Project Schedule (Gantt chart).
- System Requirements Specification (SRS) and System Architecture & Design document.
- A functional Version 1 prototype: mobile application (capture, quantity confirmation, dashboard) and web application (admin panel, analytics), demonstrated at Progress Review 1 and Mid Evaluation.
- The FreshLens Two-Tier Classifier, trained and integrated as an asynchronous microservice, demonstrated at Progress Review 2 and Final Evaluation.
- A Testing Document covering unit tests, integration tests, and: given the platform's core safety requirement: explicit multi-tenant Row-Level Security isolation tests.
- A recorded demonstration video and the Final Report, including the model's Precision/Recall metrics and a documented Version 2 roadmap.
- Final product resources (source code archive).

7. Overview of Existing Systems and Technology

7.1 Existing Similar Systems

Commercial inventory-management platforms such as Zoho Inventory [3] provide small businesses with stock tracking, low-stock alerts, and multi-warehouse reporting through a cloud-based dashboard. These systems are effective for counting and order management, but depend entirely on manual or barcode-based data entry: none offer visual freshness assessment, which is the central gap FreshLens addresses.

7.2 Related Research

Mureşan and Oltean [4] introduced the Fruits-360 dataset and demonstrated CNN-based fruit recognition with high classification accuracy; this project uses that dataset for the item-identification tier of its classifier.

Mukhiddinov et al. [1] proposed a deep-learning system using an improved YOLOv4 model (enhanced with a Mish activation function and residual network) that first identifies the produce type in an image and then classifies it as fresh or rotten. Their model achieved a higher average precision (50.4%) than baseline YOLOv4 (49.3%) and YOLOv3 (41.7%) on their own 20-category, 12,000-image dataset. This establishes the precedent for a two-stage identify-then-classify architecture, which FreshLens's Two-Tier Classifier adapts.

Fahad et al. [2] proposed VGG-16 and YOLO models that identify the object and then classify it into fresh, medium-fresh, or rotten: the same three-tier scheme FreshLens targets: reporting 82% and 84% accuracy respectively. This validates the feasibility of a three-class freshness scheme, though the specific dataset used in that study is not confirmed to be publicly released; FreshLens's own three-class training strategy is discussed under Risks and Limitations (Section 7.5).

7.3 Proposed Architecture: The FreshLens Two-Tier Classifier (FL-2TC)

Rather than a full multi-object detector, Version 1 uses two sequential classification stages on a single-product photograph:

- **Tier 1: Item Identification:** a CNN trained on the Fruits-360 dataset [5] identifies which produce type is in the frame.
- **Tier 2: Freshness Classification:** a second CNN, informed by the three-class precedent in Fahad et al. [2], classifies the identified item as Fresh, Medium, or Spoiled.

This is a deliberately scoped-down adaptation of the identify-then-classify philosophy established by Mukhiddinov et al. [1]: their pipeline uses object detection (YOLOv4) to handle multiple items per frame, which requires bounding-box-labelled training data FreshLens does not have time to collect this semester. Version 1 instead assumes one product per photograph, which is a classification problem, not a detection problem: achievable with the labelled datasets already available. Full multi-item shelf detection is documented as Version 2 future work (Section 5.3).

7.4 Planned Technologies

Based on the feasibility analysis conducted for this project, the following technologies are planned:

- **Backend API:** FastAPI [6]: async-native, and shares a language with the ML model, avoiding a cross-language boundary between the API and CNN inference code.
- **Database:** PostgreSQL [7] with native Row-Level Security, managed via Supabase [12], which also provides connection pooling (Supavisor) and authentication for the prototype.
- **Frontend (Web):** Next.js [9], for the administrative dashboard and analytics panel.
- **Mobile application:** React Native [8], for the shelf-scanning camera interface and vendor dashboard.
- **Asynchronous processing:** Redis with Celery [10], queuing and decoupling CNN inference from the API request cycle.

- **Deep Learning models:** MobileNetV3 or ResNet50 convolutional neural networks for both tiers of the classifier, trained on Fruits-360 [5] and freshness-labelled produce datasets.
- **Object storage:** Cloudflare R2 [13], an S3-compatible store chosen for zero egress fees, since dashboards repeatedly re-fetch the same images.
- **Containerisation:** Docker Compose [11] for local development and the prototype deployment.
- **Continuous Integration:** GitHub Actions [14], running linting, unit tests, and Row-Level Security isolation tests on every pull request.

Each of these choices, along with alternatives considered and rejected, is documented in full in the accompanying Technology Comparison and Feasibility Analysis. The full production-scale architecture: including the async inference pipeline, Row-Level Security design, connection pooling, observability stack, and a phased roadmap from prototype to 1M+ vendors: is documented in the accompanying Enterprise Architecture Plan.

7.5 Risks and Limitations

- **Lighting and background variation:** public training datasets use controlled studio conditions; real shelf photos under shop lighting will likely reduce classification accuracy. Data augmentation (brightness, blur, rotation) is planned to mitigate this.
- **Three-class dataset availability:** no confirmed public dataset provides Fresh/Medium/Spoiled labels at scale (Section 7.2). FreshLens will combine public binary (fresh/rotten) datasets with a small, manually curated set of intermediate-freshness images to construct the Medium class.
- **Single-item-per-photo constraint:** Version 1 does not support multi-item shelf scans; a vendor with several produce types must scan each separately (Section 5.2).
- **Manual quantity entry:** stock counts depend on the vendor entering accurate quantities at scan time; Version 1 does not verify counts automatically.
- **Generalisation to local market conditions:** training data is sourced from international datasets and may not fully represent produce varieties or presentation styles in local retail contexts; this will be evaluated during testing.

References

[1] M. Mukhiddinov, A. Muminov, and J. Cho, "Improved classification approach for fruits and vegetables freshness based on deep learning," *Sensors*, vol. 22, no. 21, p. 8192, 2022. doi: 10.3390/s22218192.

[2] L. G. Fahad, S. F. Tahir, U. Rasheed, H. Saqib, M. Hassan, and H. Alquhayz, "Fruits and vegetables freshness categorization using deep learning," *Computers, Materials & Continua*, vol. 71, pp. 5083–5098, 2022.

[3] Zoho Corporation, "Zoho Inventory: Inventory Management Software." [Online]. Available: https://www.zoho.com/inventory/ (Accessed: Jul. 4, 2026).

[4] H. Mureşan and M. Oltean, "Fruit recognition from images using deep learning," *Acta Universitatis Sapientiae, Informatica*, vol. 10, no. 1, pp. 26–42, 2018.

[5] M. Oltean, "Fruits-360 dataset." [Online]. Available: https://www.kaggle.com/datasets/moltean/fruits (Accessed: Jul. 4, 2026).

[6] FastAPI, "FastAPI documentation." [Online]. Available: https://fastapi.tiangolo.com/ (Accessed: Jul. 4, 2026).

[7] PostgreSQL Global Development Group, "Row security policies." [Online]. Available: https://www.postgresql.org/docs/current/ddl-rowsecurity.html (Accessed: Jul. 4, 2026).

[8] Meta Platforms Inc., "React Native documentation." [Online]. Available: https://reactnative.dev/ (Accessed: Jul. 4, 2026).

[9] Vercel Inc., "Next.js documentation." [Online]. Available: https://nextjs.org/docs (Accessed: Jul. 4, 2026).

[10] Redis Ltd., "Redis documentation." [Online]. Available: https://redis.io/docs/ (Accessed: Jul. 4, 2026).

[11] Docker Inc., "Docker documentation." [Online]. Available: https://docs.docker.com/ (Accessed: Jul. 4, 2026).

[12] Supabase, "Supabase documentation." [Online]. Available: https://supabase.com/docs (Accessed: Jul. 4, 2026).

[13] Cloudflare Inc., "R2 object storage documentation." [Online]. Available: https://developers.cloudflare.com/r2/ (Accessed: Jul. 4, 2026).

[14] GitHub Inc., "GitHub Actions documentation." [Online]. Available: https://docs.github.com/actions (Accessed: Jul. 4, 2026).